

NEXORA (MARKETHUB)

Complete Backend Developer Handoff & Phase-by-Phase Integration Guide

Target Audience: Backend Engineering Lead, Security Architect, DevOps Lead
Prepared by: Lead Frontend / Interaction Engineer (Team WINE)
Status: Phase 1 (Frontend) & Phase 2 (Contracts & Services) COMPLETED. Ready for backend implementation.

1. Executive Architecture Summary

NEXORA is a multi-vendor, AI-enabled commerce platform centered on **Attribute-Based Access Control (ABAC)**, zero-trust cryptographic escrow, and real-time security intelligence. Unlike basic ecommerce websites, NEXORA delivers three tightly integrated user workflows powered by a unified design system and shared backend:

- **Customer Experience:** Discover → Evaluate → Buy → Track with multi-sig escrow protection.
- **Vendor Workspace:** Manage listings → Fulfill orders → Monitor escrow payouts (Strictly isolated by merchant tenant ID).
- **Admin Sentinel:** SIEM threat telemetry → BOLA/IDOR blocking → Interactive Neo4j fraud & identity relationship graph.
- **Role-Aware AI Copilot:** Scoped by ABAC. Refuses cross-tenant data requests with explicit audit logs.

2. 13-Phase Build Roadmap & Completion Status

The exact phase progression outlined in the competition requirements is tracked below:

Phase	Focus Area	Status	Deliverables / Responsibilities
PHASE 1	Frontend UI & UX	COMPLETED	Storefront, Vendor Workspace, Admin Sentinel, Touch Mobile Feed, Modals, GSAP zooms
PHASE 2	API Contracts & Services	COMPLETED	Frozen DTOs, apiClient.ts, 5 service classes, useCart/useSecuritySentinel hooks
PHASE 3	Authentication	BACKEND NEXT	JWT / session tokens, Argon2id passwords, TOTP MFA step-up auth, role claims
PHASE 4	PostgreSQL Persistence	BACKEND NEXT	Relational schemas for users, vendors, products, escrow orders, and audit events
PHASE 5	Authorization (ABAC)	BACKEND NEXT	Tenant boundary enforcement: prevent cross-merchant leakage (BOLA / IDOR defense)
PHASE 6	Redis Layer	BACKEND NEXT	Active sessions, sliding-window rate limiting (120 req/min), temporary escrow locks
PHASE 7	Neo4j Fraud Graph	BACKEND NEXT	User-Device-IP-Order-Vendor relationships for fraud ring and sybil detection
PHASE 8	Security Middleware	BACKEND NEXT	Price tampering verification, CSP nonce enforcement, audit logging to SIEM
PHASE 9	Role-Scoped AI	BACKEND NEXT	LLM integration with ABAC system prompts and cross-tenant inquiry refusal
PHASE 10	Live API Integration	BACKEND NEXT	Toggle VITE_USE MOCKS=false; connect services to real backend endpoints
PHASE 11	Security & Attack Testing	PENDING	Simulate BOLA, CSRF, price manipulation, and test automated blocking
PHASE 12	Production Deployment	PENDING	Docker Compose (Node/Python, PostgreSQL, Redis, Neo4j, Nginx reverse proxy)
PHASE 13	Competition Demo Flow	PENDING	Customer buy → Vendor ship → Hacker probe → Admin block → Graph analysis

3. What Is Completed in the Frontend

The frontend is 100% built and compiling cleanly with Vite and TypeScript (0 errors). The directory structure is fully decoupled into services and state hooks so you do not have to touch any React/UI components to wire up your backend:

Decoupled Service Layer (src/services/):

- apiClient.ts: Axios/fetch wrapper with Bearer token injection, request ID tracking, and standardized response envelope decoding.
- productService.ts: Product search, category filtering, and product detail endpoints.
- vendorService.ts: Merchant KPIs, inventory level updates, and order fulfillment actions.
- adminService.ts: Live security alerts, probe telemetry, and Neo4j node/edge fetching.
- aiService.ts: Role-aware AI queries with built-in ABAC cross-tenant boundary refusal.
- securityService.ts: Attack vector simulation (price tamper, BOLA, XSS/CSP, CSRF).

State Encapsulation Hooks (src/hooks/):

- useCart.ts: Manages cart state, subtotal math, and client-side price tampering detection.
- useProducts.ts: Handles category switching, search debounce, and active stage selection.
- useSecuritySentinel.ts: Manages real-time SIEM event feeds and Neo4j graph nodes.

4. Backend API Contract & DTO Specifications

Your backend must implement the following standardized JSON response envelope:

```
{
  "success": true,
  "data": { ... payload ... },
  "error": { "code": "ERR_STRING", "message": "Human readable description" },
  "meta": { "timestamp": "2026-10-06T00:00:00Z", "requestId": "req_881a", "auditLogged": true }
}
```

Core Endpoints to Implement:

Method & Route	Role Required	Input Payload / Query	Expected Behavior & Protections
GET /api/v1/products	Public	?category=&q;=	Return catalog array with price, seller, and stock.
POST /api/v1/checkout	Customer	{ items, shippingAddress }	Recalculate price on server. Reject if client mutated price (422 ERR_PRICE_HASH_MISMATCH). Lock funds in escrow.
GET /api/v1/vendor/metrics	Vendor	Bearer token	Return gross revenue, active listings, escrow balance, seller score.
GET /api/v1/vendor/inventory	Vendor	Bearer token	Enforce ABAC: actor.tenantId == resource.merchantId. Return only this merchant's products.
POST /api/v1/vendor/inventory/:id/stock	Vendor	{ stock: 20 }	Verify resource ownership. Reject cross-tenant updates with 403 Forbidden.
GET /api/v1/admin/sentinel/stats	Admin	Bearer token	Return count of blocked probes (24h), active anomalies, and health status.
GET /api/v1/admin/security/events	Admin	Bearer token	Return live SIEM security logs (timestamp, actor IP, blocked endpoint, severity).
GET /api/v1/admin/security/graph	Admin	Bearer token	Return Neo4j nodes (User, Device, IP, Threat) and edges (CONNECTS_VIA, DISPATCHED).
POST /api/v1/ai/query	Any	{ prompt, role, tenantId }	Role-scoped LLM. If Vendor asks for rival sales, return ABAC refusal.

5. Database Models & Persistence Design

Your backend database stack consists of three complementary engines:

A. PostgreSQL Relational Schemas (System of Record):

```
-- 1. Users Table (Customer, Vendor, Admin)
CREATE TABLE users (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  email VARCHAR(255) UNIQUE NOT NULL,
  password_hash TEXT NOT NULL, -- Argon2id
  role VARCHAR(32) NOT NULL CHECK (role IN ('CUSTOMER', 'VENDOR', 'ADMIN')),
  mfa_enabled BOOLEAN DEFAULT FALSE,
  mfa_secret TEXT NULL,
  created_at TIMESTAMPTZ DEFAULT NOW()
);

-- 2. Vendors Table (Tenant Entity)
CREATE TABLE vendors (
  id VARCHAR(64) PRIMARY KEY, -- e.g. 'VEND-001'
  user_id UUID REFERENCES users(id) ON DELETE CASCADE,
  business_name VARCHAR(255) NOT NULL,
  kyb_status VARCHAR(32) DEFAULT 'VERIFIED',
  escrow_balance DECIMAL(12,2) DEFAULT 0.00,
  seller_score DECIMAL(4,1) DEFAULT 100.0
);

-- 3. Products Table
CREATE TABLE products (
  id VARCHAR(64) PRIMARY KEY,
  vendor_id VARCHAR(64) REFERENCES vendors(id),
  category VARCHAR(32) NOT NULL,
  name VARCHAR(255) NOT NULL,
  description TEXT NOT NULL,
  price_cents INTEGER NOT NULL CHECK (price_cents > 0),
  stock INTEGER NOT NULL DEFAULT 0,
  status VARCHAR(32) DEFAULT 'ACTIVE'
);

-- 4. Orders & Escrow Table
CREATE TABLE orders (
  id VARCHAR(64) PRIMARY KEY,
  customer_id UUID REFERENCES users(id),
  total_cents INTEGER NOT NULL,
  escrow_status VARCHAR(32) NOT NULL, -- 'ESCROW_LOCKED', 'RELEASED'
  signature_hash TEXT NOT NULL, -- SHA-256 HMAC
  created_at TIMESTAMPTZ DEFAULT NOW()
);
```

B. Redis Transient Keys:

- session:{token} → Hash storing userId, role, tenantId (TTL 24h).
- rate_limit:ip:{client_ip} → Counter sliding window (max 120 req/min). Returns HTTP 429 when breached.
- escrow:lock:{order_id} → Cryptographic escrow challenge verification state.

C. Neo4j Fraud Graph Relationships:

- Nodes: (:User), (:Vendor), (:Device), (:IP), (:Order), (:Threat)
- Edges: (User)-[:LOGGED_IN_FROM]->(Device)-[:CONNECTS_VIA]->(IP)-[:DISPATCHED]->(Threat)
- Used by Admin Sentinel to highlight multi-account fraud rings and credential stuffing.

6. Step-by-Step Integration Guide for Backend Developer

Follow these exact steps to connect your backend services to the live frontend:

Step 1 — Create your backend service (Node.js/Express, FastAPI, or Flask):

Listen on `http://localhost:5000/api/v1` (or set your port in `.env`). Enable CORS for origin `http://localhost:5173` with credentials enabled.

Step 2 — Switch Frontend from Mock to Live API:

In `BuildSecure-main/BuildSecure-main/.env`, change:

`VITE_DATA_SOURCE=api`

`VITE_API_BASE_URL=http://localhost:5000/api/v1`

The frontend service layer automatically routes all requests to your backend without changing any UI code.

Step 3 — Implement Authentication & JWT Bearer Header:

Upon login/registration, return `{ token: '...', user: { id, role, tenantId } }`. The frontend `apiClient.ts` automatically stores the token and attaches `Authorization: Bearer <token>` on subsequent calls.

Step 4 — Implement ABAC Middlewares:

Protect `/api/v1/vendor/*`: Check that `req.user.tenantId === req.params.merchantId`. If mismatched, return HTTP 403 with code `ERR_ABAC_OWNERSHIP_FAILURE`. Log the event to your security SIEM table.

Step 5 — Connect AI & Neo4j:

For POST `/api/v1/ai/query`, inspect the caller role. If role is `VENDOR` and query contains keywords regarding competitors or cross-merchant records, inject the strict system refusal prompt. For GET `/api/v1/admin/security/graph`, execute a Cypher query returning nodes and relationships.

7. Competition Narrative Checklist (Phase 13 Demo)

This is the exact sequence to demonstrate to judges to win the competition:

- 1. Customer Flow:** Browse catalog, select product, examine verified merchant badge, view escrow assurance, and add to bag.
- 2. Attack Simulation:** Disclose an attacker modifying client cart payload to INR 1.00 (■1) → Backend detects hash mismatch, rejects with HTTP 422, and logs event to SIEM.
- 3. Vendor Flow:** Vendor logs in, inspects inventory, edits stock, and queries AI Copilot: *'Show me sales for VEND-002'* → AI rejects via ABAC (Zero tenant data leakage).
- 4. Admin Sentinel Flow:** Admin opens command center, observes live 1,429 blocked probes counter, inspects real-time SIEM event, and explores Neo4j fraud ring graph.